B2B LOGISTICS

Platform Documentation

Complete System Architecture & Module Reference

17 Modules • Backend + Frontend

17 Backend Modules • 10 Frontend Modules

Generated: July 22, 2026

Table of Contents

MODULE 1

Backend Configuration & Server Setup

OVERVIEW

This module sets up the Express.js backend server. It defines the main server.js entry point, loads environment variables via dotenv, connects to MongoDB using Mongoose, and registers all API route handlers. CORS is configured to allow the React frontend to communicate with the backend. JSON body parsing is enabled with a 10 MB limit for large payloads. A health-check endpoint at /api/health is exposed for monitoring. Error handling middleware catches all unhandled errors globally. A scheduled cron job (overdueSweep) runs daily at midnight to automatically flag manifests that have exceeded their delivery window.

KEY FILES

`server.js` • `config/db.js` • `config/cors.js` • `.env` • `package.json`

FEATURES & CAPABILITIES

- Express.js REST API server on port 5000
- MongoDB Atlas connection via Mongoose
- CORS configured for frontend origin
- Global error handler middleware
- Daily cron job for overdue delivery detection
- JWT-based authentication infrastructure
- Environment-based configuration (.env)

MODULE 2

Backend Middleware

OVERVIEW

This module contains three essential middleware functions. The auth middleware verifies JWT tokens from the Authorization header, decodes the payload, and attaches the user object to the request. The roleGuard middleware takes an array of allowed roles and returns a 403 Forbidden if the authenticated user does not match. The errorHandler middleware catches all errors, logs them, and returns a structured JSON error response with status code and message. These middleware are applied to route definitions to protect endpoints and enforce role-based access control across the application.

KEY FILES

`middleware/auth.js` • `middleware/roleGuard.js` • `middleware/errorHandler.js`

FEATURES & CAPABILITIES

- JWT token verification and user attachment
- Role-based access control (admin, client, driver, executive)
- Centralized error handling with structured responses
- Applied to protected API routes
- Returns proper HTTP status codes on failure

MODULE 3

Database Models

OVERVIEW

This module defines all Mongoose schemas and models for the application. The User model stores authentication credentials, role, profile info (name, email, company, phone), contract rate for invoicing, and license number for drivers. Passwords are hashed with bcrypt before saving. The Vehicle model tracks registration, make/model/year, weight and volume capacity, operational status, assigned driver, and fuel efficiency. The Manifest model is the core entity — it holds tracking ID, client/driver/vehicle references, cargo details (description, weight, volume, item count, hazmat flag), routing with origin/destination coordinates, status with a full timeline history, and pickup/delivery scheduling fields. The Invoice model tracks billing with line items, amounts, status (Pending/Paid/Overdue/Cancelled), and due dates. The Notification model stores alerts with type, read status, and optional manifest reference.

KEY FILES

`models/User.js` • `models/Vehicle.js` • `models/Manifest.js` • `models/Invoice.js` • `models/Notification.js`

FEATURES & CAPABILITIES

- User model with bcrypt password hashing and role enum
- Vehicle model with capacity fields and status tracking
- Manifest model with cargo, routing, status timeline, and scheduling
- Invoice model with line items and payment status
- Notification model with type classification and read tracking
- Database indexes on critical query fields for performance
- Timestamps (createdAt/updatedAt) on all models

MODULE 4

Backend Routes

OVERVIEW

This module defines all Express.js route mappings for the application. Each route file groups related endpoints together and applies appropriate middleware (auth + roleGuard). The auth routes handle login, registration, and current user retrieval. User routes provide CRUD operations for managing platform users. Vehicle routes expose listing, creation, updating, status changes, and statistics. Manifest routes cover the full lifecycle — creation, listing (with filters), driver-specific views, assignment, trip start, status updates, completion, and cancellation. Invoice routes handle listing, generation from manifests, payment marking, and statistics. Analytics routes provide fleet utilization, route efficiency, monthly capacity, and delivery performance data.

KEY FILES

`routes/auth.routes.js` • `routes/user.routes.js` • `routes/vehicle.routes.js` • `routes/manifest.routes.js` • `routes/invoice.routes.js` • `routes/analytics.routes.js` • `routes/notification.routes.js`

FEATURES & CAPABILITIES

- RESTful route design with proper HTTP methods
- Auth routes: login, register, getMe
- Vehicle routes: CRUD, available list, stats
- Manifest routes: full lifecycle management
- Invoice routes: list, generate, pay, stats
- Analytics routes: fleet, route, monthly, performance
- Role-based middleware on every protected route

MODULE 5

Backend Controllers

OVERVIEW

This module contains all request handler logic for every API endpoint. The auth controller manages user registration with email uniqueness checks, password hashing, and JWT token generation. Login validates credentials and returns a token. The user controller provides paginated listing with role filtering. The vehicle controller handles CRUD with automatic status management and availability queries. The manifest controller is the most complex — it validates cargo against vehicle capacity, calculates route distance using the Haversine formula, manages the full status lifecycle with timeline recording, supports driver assignment, and triggers invoice generation upon delivery. The invoice controller calculates amounts based on distance and client contract rates. The analytics controller aggregates data across collections for fleet utilization percentages, monthly shipment/revenue trends, route on-time percentages, and overall delivery performance breakdown.

KEY FILES

`controllers/auth.controller.js` • `controllers/user.controller.js` • `controllers/vehicle.controller.js` • `controllers/manifest.controller.js` • `controllers/invoice.controller.js` • `controllers/analytics.controller.js`

FEATURES & CAPABILITIES

- JWT token generation with configurable expiry
- Password hashing with bcrypt (12 rounds)
- Paginated and filtered data queries
- Automatic route distance calculation (Haversine)
- Vehicle capacity matching for cargo
- Status timeline recording on every state change
- Invoice auto-generation on delivery completion
- Aggregate analytics across multiple collections

MODULE 6

Backend Services, Cron Jobs & Email

OVERVIEW

This module provides utility services and scheduled jobs. The routeCalculator service implements the Haversine formula to compute distance between two GPS coordinates and estimates travel duration at 60 km/h average. The capacityMatcher service queries available vehicles sorted by capacity to find ones that fit a manifest's weight and volume requirements. The invoiceGenerator service calculates shipping cost based on distance, client contract rate, and cargo weight, then creates the invoice record with a 30-day payment window. The emailService wraps nodemailer for sending HTML emails and includes a specific overdue notification template. The overdueSweep cron job runs daily at midnight, finds all In-Transit or Assigned manifests past their delivery window, marks them as Delayed, records the status change, creates user notifications, and sends email alerts.

KEY FILES

`services/routeCalculator.js` • `services/capacityMatcher.js` • `services/invoiceGenerator.js` • `services/emailService.js` • `cron/overdueSweep.js`

FEATURES & CAPABILITIES

- Haversine distance calculation between coordinates
- Travel time estimation at 60 km/h average speed
- Vehicle capacity matching (weight + volume)
- Automatic invoice generation with line items
- SMTP email delivery via nodemailer
- Daily midnight cron job for overdue detection
- Automatic status escalation to Delayed
- Notification + email alerts on overdue

MODULE 7

Database Seed Script

OVERVIEW

This module provides a comprehensive seed script that populates the database with realistic demo data. It creates four user accounts (admin, two clients, two drivers, and one executive) with pre-hashed passwords for instant login. It seeds 8 vehicles of various types with different capacities and statuses. It generates 15 sample manifests spread across different statuses (Pending, Assigned, In-Transit, Delivered, Delayed) with realistic cargo descriptions, origin/destination coordinates in the US, and proper scheduling dates. The seed script is idempotent — it clears existing data before seeding to prevent duplicates. Demo credentials are printed to console after seeding completes.

KEY FILES

`seeds/seed.js`

FEATURES & CAPABILITIES

- Creates demo accounts for all 4 roles
- Pre-seeded passwords (no hashing needed at runtime)
- 8 vehicles with varying capacities
- 15 manifests across all status types
- Realistic US-based origin/destination coordinates
- Idempotent — safe to re-run
- Console output of demo credentials

MODULE 8

Frontend Configuration

OVERVIEW

This module sets up the React frontend build system. It uses Vite as the development server and bundler for fast HMR and builds. React 18 provides the UI framework. Tailwind CSS v4 is integrated via the Vite plugin for utility-first styling. The package.json includes all production dependencies: Redux Toolkit for state management, React Router for navigation, Axios for HTTP, React Hot Toast for notifications, Mapbox GL for map rendering, and Recharts for analytics charts. The vite.config.js configures the dev server on port 5173 and enables both React and Tailwind plugins. The .env file holds the API base URL and Mapbox token.

KEY FILES

`package.json` • `vite.config.js` • `index.html` • `src/styles/globals.css` • `.env`

FEATURES & CAPABILITIES

- Vite dev server with hot module replacement
- React 18 with strict mode
- Tailwind CSS v4 via Vite plugin
- Redux Toolkit + React Query for data management
- React Router v6 for client-side routing
- Axios with interceptors for API communication
- Mapbox GL for map rendering
- Recharts for data visualization

MODULE 9

Frontend State Management (Redux Toolkit)

OVERVIEW

This module defines the Redux store and all state slices using Redux Toolkit. The store combines four slices: auth, manifest, vehicle, and ui. The authSlice manages user session state with async thunks for login (which stores JWT in localStorage) and loadUser (which validates existing tokens). It tracks loading/error states. The manifestSlice holds the manifest list, selected manifest, and filter/pagination state. The vehicleSlice mirrors this for vehicle data. The uiSlice controls sidebar visibility and modal state. All slices use Redux Toolkit's Immer-based reducers for immutable state updates with minimal boilerplate.

KEY FILES

`store/store.js` • `store/authSlice.js` • `store/manifestSlice.js` • `store/vehicleSlice.js` • `store/uiSlice.js`

FEATURES & CAPABILITIES

- Centralized Redux store with 4 slices
- Async thunks for login and user loading
- JWT persistence in localStorage
- Immutable state updates via Immer
- UI slice for sidebar and modal control
- Filter/pagination state for manifest listing
- Loading and error state tracking per slice

MODULE 10

Frontend API Services, Hooks & Utilities

OVERVIEW

This module provides the frontend's data layer and helper utilities. The api.js file creates a configured Axios instance with a base URL from environment variables, attaches JWT tokens to every request via interceptor, and handles 401 responses by clearing the token and redirecting to login. Individual API service files (authApi, manifestApi, vehicleApi, invoiceApi, analyticsApi, notificationApi) wrap each backend endpoint in clean function calls. Custom hooks include useAuth (returns current user, role, and auth state from Redux), useRecovery (restores trip timer state from localStorage for page reload resilience), useLocalStorage (generic localStorage hook with JSON serialization), and useDebounce (delays value updates for search inputs). Utility files provide formatting functions (dates, currency, weight, volume, duration, elapsed time), validation helpers (email, phone, required fields, positive numbers), and constants for roles, statuses, and status color mappings.

KEY FILES

`services/api.js` • `services/authApi.js` • `services/manifestApi.js` • `services/vehicleApi.js` • `services/invoiceApi.js` • `services/analyticsApi.js` • `services/notificationApi.js` • `hooks/useAuth.js` • `hooks/useRecovery.js` • `hooks/useLocalStorage.js` • `hooks/useDebounce.js` • `utils/constants.js` • `utils/formatters.js` • `utils/validators.js`

FEATURES & CAPABILITIES

- Axios instance with JWT auto-attachment
- 401 response auto-redirect to login
- Clean API wrapper functions for all endpoints
- useAuth hook for current user/role state
- useRecovery hook for trip timer persistence
- Date, currency, weight, volume formatters
- Email, phone, required field validators
- Status-to-color mapping constants

MODULE 11

Frontend App Shell, Router & Layout

OVERVIEW

This module defines the application entry point, routing structure, and layout components. The main.jsx renders the React app wrapped in Redux Provider, BrowserRouter, and React Hot Toast Toaster. The App.jsx component loads the user on mount if a token exists and defines all routes using React Router v6 nested routes. ProtectedRoute checks authentication and wraps protected pages in the AppShell layout. Unauthenticated users are redirected to /login. The AppShell component creates a sidebar + topbar + main content area using a flex layout. The Sidebar component renders role-based navigation links — admin sees Dashboard/Fleet/Create/Live, client sees Dashboard/Order/Track/Invoices, driver sees My Deliveries, executive sees Analytics. The Topbar shows the user's name and a logout button.

KEY FILES

`main.jsx` • `App.jsx` • `ProtectedRoute.jsx` • `AppShell.jsx` • `Sidebar.jsx` • `Topbar.jsx`

FEATURES & CAPABILITIES

- Role-based route definitions
- Protected route wrapper with auth check
- Responsive sidebar (hides on mobile)
- Role-specific navigation menus
- User info display in topbar
- One-click logout with state cleanup
- Automatic user loading from stored token

MODULE 12

Shared UI Components

OVERVIEW

This module contains reusable UI components used across all pages. StatusBadge renders a colored pill label for any status string, using the status-to-color mapping from constants. StatCard displays a metric with an icon, title, and value in a card layout — used on all dashboard pages. ProgressStepper shows a 4-step progress bar (Pending → Assigned → In-Transit → Delivered) with filled/unfilled circles and connecting lines based on current status. DataTable is a generic table component that accepts column definitions and row data, with click handlers for row interaction and an empty state message. ConfirmModal is a reusable confirmation dialog with title, message, confirm/cancel buttons, and a dark backdrop overlay.

KEY FILES

`components/shared/StatusBadge.jsx` • `components/shared/StatCard.jsx` • `components/shared/ProgressStepper.jsx` • `components/shared/DataTable.jsx` • `components/shared/ConfirmModal.jsx`

FEATURES & CAPABILITIES

- StatusBadge: colored status pills with automatic color mapping
- StatCard: metric display with icon, title, and value
- ProgressStepper: 4-step visual progress indicator
- DataTable: generic sortable table with click handlers
- ConfirmModal: reusable confirmation dialog with backdrop
- All components are prop-driven and reusable
- Consistent styling across the application

MODULE 13

Login Page

OVERVIEW

This module implements the authentication login page. It presents a centered card with email and password inputs, a submit button, and demo account credentials. On form submission, it dispatches the Redux loginUser async thunk which calls the backend /auth/login endpoint. On success, the JWT token is stored in localStorage and the user is redirected to their role-specific dashboard (admin → /admin/dashboard, client → /client/dashboard, driver → /driver/dashboard, executive → /executive/analytics). Error messages from the backend are displayed via React Hot Toast notifications. The loading state disables the submit button and shows "Signing in..." text. Demo credentials are displayed below the form for easy testing.

KEY FILES

`pages/Login.jsx`

FEATURES & CAPABILITIES

- Email and password form with validation
- Redux-powered async login dispatch
- Role-based redirect after successful login
- Error toast notifications
- Loading state with disabled button
- Demo account credentials displayed
- Dark background with centered card design

MODULE 14

Admin Pages

OVERVIEW

This module provides four admin-only pages. The AdminDashboard shows four stat cards (total manifests, vehicles, pending orders, delayed) and a table of the 5 most recent manifests with tracking ID, client name, status badge, and creation date. The FleetMonitor page displays vehicle fleet status with filterable tabs (All/Available/In-Transit/Maintenance), stat summary cards, and a detailed table showing registration number, vehicle description, capacity, status, and assigned driver. The ManifestCreate page is a multi-section form that allows admins to create new manifests — Section 1 selects a client and sets pickup/delivery schedule, Section 2 captures cargo details (description, weight, volume, item count, hazmat checkbox), and Section 3 defines origin/destination addresses. On submit, it calls the manifest API and redirects to the dashboard. The LiveOperations page is the real-time operations center — it shows manifests filtered by status, each card displaying tracking ID, route, client/driver info, and a live elapsed trip timer (updates every second via setInterval). Admins can assign drivers/vehicles to pending manifests (via a modal with dropdown selects), start transit, mark delivered, flag delayed, or cancel — all with inline action buttons.

KEY FILES

`pages/admin/AdminDashboard.jsx` • `pages/admin/FleetMonitor.jsx` • `pages/admin/ManifestCreate.jsx` • `pages/admin/LiveOperations.jsx`

FEATURES & CAPABILITIES

- AdminDashboard: 4 stat cards + recent manifests table
- FleetMonitor: vehicle grid with status filter tabs
- ManifestCreate: 3-section form (client, cargo, routing)
- LiveOperations: real-time manifest cards with live timers
- Driver/vehicle assignment via modal dropdown
- Inline status update actions (transit, deliver, delay, cancel)
- HAZMAT warning badges on hazardous shipments

MODULE 15

Client Pages

OVERVIEW

This module provides four client-facing pages. The ClientDashboard shows personal stats (total shipments, active, delivered, pending invoices), recent shipments list with status badges, and pending invoices with amounts and due dates — each section links to its full page. The PlaceOrder page is a clean order form where clients enter cargo details (description, weight, volume, item count, hazmat flag), pickup and delivery addresses, and preferred schedule (pickup datetime and delivery window close). On submit it creates a manifest assigned to the logged-in client. The TrackShipment page provides a split-pane view — the left panel lists all client shipments searchable by tracking ID, and the right panel shows full details when a shipment is selected, including the ProgressStepper timeline, origin/destination addresses, weight/volume, driver info, and the full status timeline with timestamps and notes. The ClientInvoices page displays invoice stats (total billed, paid, pending, overdue amounts), filterable tabs, and a table with invoice number, amount, status badge, issue/due dates, and a "Mark Paid" action button for pending invoices.

KEY FILES

`pages/client/ClientDashboard.jsx` • `pages/client/PlaceOrder.jsx` • `pages/client/TrackShipment.jsx` • `pages/client/ClientInvoices.jsx`

FEATURES & CAPABILITIES

- ClientDashboard: personal stats + recent shipments + pending invoices
- PlaceOrder: full order form with cargo, addresses, and schedule
- TrackShipment: searchable split-pane with progress stepper + timeline
- ClientInvoices: financial stats + filterable invoice table
- Status timeline with reverse chronological history
- Mark Paid action for pending invoices
- Direct links between dashboard sections and full pages

MODULE 16

Driver Pages

OVERVIEW

This module provides two driver-focused pages. The DriverDashboard shows two sections — Active Deliveries (manifests with Assigned or In-Transit status) and Recent History (Delivered/Delayed manifests). Each active delivery card displays the tracking ID, route, cargo summary, status badge, HAZMAT warning if applicable, a live elapsed trip timer, and an "Open" button that navigates to the active delivery detail view. The ActiveDelivery page is the main work screen for drivers during a delivery. It shows a back button, manifest header with tracking ID and status, a ProgressStepper, and a large TripTimer component that displays hours:minutes:seconds in a prominent indigo card — the timer uses localStorage via the useRecovery hook so it survives page reloads and browser crashes. Below the timer, two info cards show the route (origin → destination with estimated distance) and cargo details (weight, volume, item count, hazmat flag). A schedule card displays pickup time, delivery window close, and actual delivery time if completed. Action buttons adapt to status: Assigned shows "Start Trip", In-Transit shows "Report Delay" and "Complete Delivery". On delivery completion, the localStorage timer is cleared. An activity log at the bottom shows the full status timeline in reverse order.

KEY FILES

`pages/driver/DriverDashboard.jsx` • `pages/driver/ActiveDelivery.jsx`

FEATURES & CAPABILITIES

- DriverDashboard: active deliveries + history table
- Live trip timer with HH:MM:SS display
- Timer recovery from localStorage (survives reload/crash)
- ActiveDelivery: full delivery detail view
- Start Trip → In-Transit → Complete Delivery workflow
- Report Delay action during transit
- Route card with origin/destination/distance
- Cargo card with weight/volume/items/hazmat
- Activity log with reverse chronological timeline

MODULE 17

Executive Analytics Page

OVERVIEW

This module provides a data-driven analytics dashboard for executive users. It fetches data from four analytics API endpoints (fleet utilization, route efficiency, monthly capacity, delivery performance) using Promise.allSettled so partial failures don't break the page — if the backend isn't available, it falls back to realistic mock data. The page displays four KPI metric cards at the top (total shipments, revenue, fleet utilization percentage, on-time rate) with period-over-period change indicators. Below the metrics, five Recharts visualizations are arranged in a 2-column grid: (1) an AreaChart showing monthly shipments with a Line overlay for revenue on a dual Y-axis, (2) a donut PieChart showing fleet status distribution (Available/In-Transit/Maintenance) with inner radius for the donut effect, (3) a grouped BarChart comparing on-time vs delayed percentages by route corridor, (4) a standard PieChart breaking down delivery performance (On-Time/Delayed/Cancelled) with custom colors, and (5) a full-width LineChart showing monthly shipment trends. All charts use ResponsiveContainer for auto-sizing and include tooltips, legends, and grid lines.

KEY FILES

`pages/executive/ExecutiveAnalytics.jsx`

FEATURES & CAPABILITIES

- 4 KPI cards with period-over-period change indicators
- AreaChart: monthly shipments + revenue (dual Y-axis)
- Donut PieChart: fleet status distribution
- BarChart: route efficiency by corridor (on-time vs delayed)
- PieChart: delivery performance breakdown
- LineChart: monthly shipment trend
- Graceful fallback to mock data if API unavailable
- Promise.allSettled for partial failure resilience
- All charts responsive via Recharts ResponsiveContainer

End of Document

B2B Logistics Platform • 2026